Name : Endang Kosasih

Position : Senior Software Engineer

C# Language – Visual Studio 2022

A . Coding Competency Test

Path : folder [CodingCompetensiTest.sln]

B. Object Oriented Test

Path : folder [Formulatrix.sln]

C. Software Comprehension Test

Path : folder [FormulatrixIFrame.sln]

C.1 What would you say about code ?

This code demonstrates a basic understanding of event-driven programming and separation of duties, but fails to address the critical characteristics of camera data (high-frequency and shared memory). It is extremely inefficient in memory usage and has serious logic bugs in its mathematical calculations.

C.2 What sorts of problems does this code have ?

- Memory Corruption:

Inside FrameGrabber, the code performs a Marshal.Copy on the _buffer, then sends that _buffer into the Frame object. The problem is, the Frame object doesn't copy the data, it just holds a reference to it. Immediately afterward, bufferedFrame.Dispose() is called. As a result, when OnTimerElapsed tries to access the data, the object is already in the disposed state or the data has changed because the buffer was reused by the native library.

- Race Condition & Thread Safety:

Access to _receivedFrames (a Queue) occurs on two different threads: the camera callback thread (when Enqueue) and the timer thread (when Dequeue). Queue<T> in C# is not thread-safe by default, so the application is at risk of crashing under high load.

- Kesalahan fatal perhitungan rata-rata

The sum variable is defined as an int. However, the total pixel values of a single frame (e.g., 1920x1080) would far exceed the maximum int limit (2.1 billion), resulting in an Integer Overflow. Furthermore, the division result is rounded down because it uses an int data type, not a double.

- Frame Rate Inconsistency :

Menggunakan Timer dengan interval 33ms (1000/30) untuk mengambil data dari antrean tidak menjamin sinkronisasi dengan kamera. Jika proses pengolahan lebih lambat dari 33ms, antrean akan terus membengkak (memory leak). Jika lebih cepat, timer akan sering menemukan antrean kosong.

C.3 How can this code be improved ?

Use Deep Copy or Array Pooling:

Each received frame must be copied into a new, unique array before being queued, to avoid being overwritten by subsequent frames from the camera. Use ArrayPool<byte> to avoid overloading the Garbage Collector due to allocating thousands of arrays per second.

Use ConcurrentQueue:

Replace Queue<Frame> with System.Collections.Concurrent.ConcurrentQueue<Frame> to ensure data safety between threads without the need for complex manual locking.

Improve Calculation Data Types:

Use the long data type for the sum variable to avoid overflow, and ensure division results in a double before sending it to _reporter.Report().

Remove Timers (Push Model):

Instead of using timers to "pull" data, process it directly in a background thread or use ActionBlock<T> from the Dataflow TPL. As soon as a frame is received, send it to the worker thread for calculation, so the value is reported in real time without timer delays.

Loop Optimization:

Use Span<byte> or SIMD instructions to calculate pixel averages to significantly reduce CPU usage at high resolutions.